

Budgie – Service Layers

By Ella Huebotter

November 1, 2026

SWDV691

1. Introduction

In my app, the React UI will communicate with the PostgreSQL database through the service layer. The backend service layer will consist of:

1. The route layer – this layer will look at the incoming HTTP method request and the URL endpoint to direct the request to the right controller.
2. The controller layer – this layer takes the incoming request and determines what to send back.
3. The service layer – this layer takes care of the business logic (hash password, create trip, calculate remaining budget, check category limits, etc.)

Budgie will use Node.js and Express for the backend controller REST API to communicate. All business logic will go through HTTP endpoints.

*The stretch feature will allow users to add a second, shared user to a trip by entering the second users email when creating or editing a trip. On shared trips the user and shared user will be able to add expenses and see the trip dashboard. On the All Expenses page the user that added the expense will be shown. The trip will show up for both users on the My Trips page. The budget and category limits will stay the same for both users. Only the owner/creator of the trip will be able to change the budget and category limits.

2. Service Endpoints

Budgie will be deployed using Render. When it is deployed the base URL will look similar to 'https://budgie-api.onrender.com/api'

2.1 Authentication

POST /api/auth/register

Purpose: Create a new user account and insert a new row into the user table. This works for all user personas the first time they use Budgie

Persona impact: Creates and adds a new user record so personas can own and create trips and expenses

Request:

POST /api/auth/register

```
{
  "email": "shaunap@gmail.com",
  "password": " MyTrip2026!",
  "name": "Shauna"
}
```

Success response (201 Created):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "user_id": 1,
    "email": "shauna@email.com",
    "name": "Shauna"
  }
}
```

Error responses:

400 Bad Request – missing fields

{ "error": "Email, password, and name are required" }

409 Conflict – email already registered
{ "error": "Email already in use" }

POST /api/auth/login
Purpose: Authenticate an existing user. Compares the submitted password to the stored password_hash (bcrypt). Returns a token that is used for subsequent requests to validate user logged in.

Persona impact: Does not change data. It reads the users table to verify credentials so the user can access their account info like trips and expenses.

Request:

POST /api/auth/login
{
  "email": "shaunap@gmail.com",
  "password": " MyTrip2026!"
}
Success response (200 OK):
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "user_id": 1,
    "email": "shaunap@gmail.com",
    "name": "Shauna"
  }
}
Error responses:

400 Bad Request

{ "error": "Email and password are required" }

401 Unauthorized – wrong email or password

{ "error": "Invalid email or password" }

2.2 Trips

GET /api/trips

Purpose: Return all trips the logged-in user has created. Used on the MyTrips page. *For the stretch feature, it will return all trips the logged-in user has created or has been shared on. Shared trips will show up for both people.

Persona impact: Reads trips for the user. Lets the personas see a list of all trips they have created. Allows personas, like Nancy, compare trips to each other. *For the stretch feature it allows Daniel and his wife to see the shared trip.

Request:

GET /api/trips

Authorization: Bearer <token>

Success response (200 OK):


```
{
  "trip_id": 1,
  "trip_title": "2026 Fall Trip",
  "remaining": 869.00,
  "percent_used": 65.2
}
```


Error responses:

401 Unauthorized – missing or invalid token

{ "error": "Authentication required" }

POST /api/trips

Purpose: Create a new trip and add optional category spending limits. Inserts into trip table and category_limit table. *For the stretch feature, users also have the option to add a shared user to the trip. Shared_user_id will start as null.

Persona impact: Adds/creates a new trip for the users. Category limits let Devin and Nancy add spending limits to stay on budget. *The stretch feature allows Daniel to share the trip with his wife.

Request:

POST /api/trips

Authorization: Bearer <token>

{

"trip_title": "California Trip",
"total_budget": 1000.00,
"category_limits": { "category": "Food", "limit_amount": 300.00 }
}
Success response (201 Created):

{
"trip_id": 2,
"trip_title": "California Trip",
"total_budget": 1000.00,
"category_limits": { "category_id": 1, "category": "Food", "limit_amount": 300.00 },
"shared_user_id": null
}
Error responses:

400 Bad Request – missing required fields or invalid dates

{ "error": "Title and total_budget are required" }

401 Unauthorized

{ "error": "Authentication required" }

Get /api/trips/:trip_id

Purpose: Return selected trip with the updated computed dashboard data (remaining budget, spent, category breakdown, limit status). Powers the trip dashboard page. *For the stretch feature, it will allow access to the trip for the user and shared user of the trip.

Persona impact: Reads the trip, expenses, and limits for user to see. Calculates remaining budget and category percent totals so personas like Devin can see how much they have left and whether they are over a category limit. Shauna can all see the cost breakdown pie-chart data. *For the stretch feature it allows Daniel's wife to see the shared trip.

Request:

GET /api/trips/1

Authorization: Bearer <token>

Success response (200 OK):

{

```
  "trip_id": 1,
  "trip_title": "2026 Fall Trip",
  "total_budget": 2500.00,
  "total_spent": 1631.00,
  "remaining": 869.00,
  "percent_used": 65.2,
  "over_budget": false,
   "category_breakdown": [
      { "category": "Food", "spent": 420.00, "percent": 25.8, "limit": 500.00, "over_limit": false },
      { "category": "Lodging", "spent": 1131.00, "percent": 69.3, "limit": null, "over_limit": false },
      { "category": "Transport", "spent": 80.00, "percent": 4.9, "limit": null, "over_limit": false }
   ],
   "category_limits": [
   {“catgory_limit_id”: 1, "category": "Food", "limit_amount": 500.00 }
   ]
}
```


Error responses:

401 Unauthorized


```
{ "error": "Authentication required" }
```


403 Forbidden – Not owner or shared user


```
{ "error": "Access denied" }
```


404 Not Found


```
{ "error": "Trip not found"}
```


PUT /api/trips/:trip_id

Purpose: Update trip details and/or category limits (Edit Trip). *For the stretch feature only the owner of the trip can edit this.

Persona impact: Updates the trip row and the category_limit rows.

Request:

PUT /api/trips/1

Authorization: Bearer <token>


```
{
   "title": "2026 Fall Trip",
   "total_budget": 2600.00,
```

```
   "category_limits": [
   { "category": "Food", "limit_amount": 600.00 }
   ]
}
```

Success response (200 OK): same form as GET /api/trips/:id with updated values

```
{
  "trip_id": 1,
  "trip_title": "2026 Fall Trip",
  "total_budget": 2600.00,
  "total_spent": 1631.00,
  "remaining": 869.00
   "category_breakdown": [
      { "category": "Food", "spent": 420.00, "percent": 25.8, "limit": 600.00, "over_limit": false },
      { "category": "Lodging", "spent": 1131.00, "percent": 69.3, "limit": null, "over_limit": false },
      { "category": "Transport", "spent": 80.00, "percent": 4.9, "limit": null, "over_limit": false }
   ],
   "category_limits": [
   {“catgory_limit_id”: 1, "category": "Food", "limit_amount": 500.00 }
   ]
}
```

Error responses: 400, 401, 403, 404 (same as GET /api/trips/:id)

POST /api/trips/:trip_id/share *Stretch Feature

This endpoint is for the stretch feature of sharing a trip with another user.

Purpose: Share the trip with another user by email. Sets shared_user_id in the Trip table. Only the user who created the trip can call this. A message will then show that says “Added <shared_user_name> to <title>”

Persona impact: Lets Daniel or any owner invite one other person so both can add expenses against the same budget. Allows him and his wife to add expenses to the same trip. Adds his wife as the shared user in the trip table.

Request:

POST /api/trips/1/share

Authorization: Bearer <token>

{

```
  "email": "wife@gmail.com"
}
```
Success response (200 OK):

```
{
  "trip_id": 1,
  "title": "2026 Fall Trip",
  "shared_user_id": 2,
  "shared_user_name": "Meagan"
}
```
Error responses:

400 – email not found or invalid

{ "error": "No user found with that email" }

403 – not the owner

{ "error": "Only the trip owner can share this trip" }

404 – trip not found

2.3 Expenses

GET /api/trips/:trip_id/expenses

Purpose: List all of the expenses for a selected trip. This supports the All Expenses page. *For the stretch feature the owner and shared user can see this. This will also show the user that entered the expense.

Persona impact: Reads the expense rows in the Expense table. Allows Shauna and Nancy to review every logged purchase they input. *For the stretch feature it allows Daniel and his wife to see who added the expense by reading the added_by_user_id row in the Expense table

Request:

GET /api/trips/1/expenses

Authorization: Bearer <token>

Success response (200 OK):
```
[
  {
    "expense_id": 1,
    "amount": 10.00,
    "category": "Food",
```

```
      "title": "Dinner with mom",
      "description": “Dinner at Outback with mom. Ordered steak and potatoes.”,
      "date": "2026-10-27"
      *"added_by_user_id": 4,
   *"user": "Daniel"
   },
{
      "expense_id": 1,
      "amount": 1131.00,
      "category": "Lodging",
      "title": "Hotel",
      "description": "Marriot Hotel",
      " date": "2026-10-27"
      *"added_by_user_id": 5,
      *"user": "Meagan"
   }
]
```


Errors:

401 Unauthorized

{ "error": "Authentication required" }

403 Forbidden – trip belongs to another user and not shared

{ "error": "Access denied" }

404 Not Found

{ "error": "Expense not found"}

POST /api/trips/:tripId/expenses

Purpose: Log a new expense. Inserts into the expense table and the dashboard numbers will update. *For the stretch feature both the owner and shared user can add an expense.

Persona impact: Adds/creates an expense row to the expense table. This is the main action the personas will use while traveling. It will trigger recalculation of the dashboard for the budget and category percentages. *For the stretch feature Daniel and his wife can add expenses and they will be logged in the expense table. The added_by_user_id will be set to the logged in user.

Request:

POST /api/trips/1/expenses

Authorization: Bearer <token>

{

"amount": 10.00,
"category": "Food",
"title": "Dinner with mom",
"description": “Dinner at Outback with mom. Ordered steak and potatoes.”,
"date": "2026-10-27"
}

Success response (201 Created):

{
"expense_id": 1,
"trip_id": 1,
"amount": 10.00
"category": "Food",
"title": "Dinner with mom",
"description": “Dinner at Outback with mom. Ordered steak and potatoes.”,
"date": "2026-10-27"
*"user": "Shauna"
}

Error responses:

400 Bad Request – missing amount or category

{ "error": "Amount and category are required" }

400 Bad Request – invalid amount

{ "error": "Amount must be a positive number" }

401, 403, 404 same as GET /api/trips/:id/expenses above

PUT /api/expenses/:id

Purpose: Edit an existing logged expense (amount, category, title, etc.).

Persona impact: Updates one row in expenses table. Dashboard updates accordingly.

Request: PUT /api/expenses/1

Authorization: Bearer <token>

{
"amount": 11.00,
"category": "Food",
"title": "Dinner with mom",
"description": “Dinner at Outback with mom. Ordered steak and potatoes.”,
"date": "2026-10-27"
}
Success: 200 OK

{
"expense_id": 1,
"trip_id": 1,
"amount": 10.00
"category": "Food",
"title": "Dinner with mom",
"description": “Dinner at Outback with mom. Ordered steak and potatoes.”,
"date": "2026-10-27"
*"user": "Shauna"
}
Errors: 400, 401, 403, 404 same as above

DELETE /api/expenses/:id

Purpose: Removes an expense. The dashboard resets the remaining budget.

Persona impact: Removes one row in the expenses table. Personas can use this if they need to delete an expense. *For the stretch feature this will remove the expense for both users from the Expense table.

Request: DELETE /api/expenses/1 + Authorization

Success: 204 No Content

Errors: 401, 403, 404 same as GET /api/trips/:id/expenses above

3.Endpoints Diagram

UI Page	Endpoints
Login	POST /api/auth/login
Create account	POST /api/auth/register
My Trips	GET /api/trips
Add new trip	POST /api/trips

Trip Dashboard	GET /api/trips/:trip_id (loads remaining budget, spent, category percent data, limit status)
All expenses	GET /api/trips/:trip_id/expenses
Add new expense	POST /api/trips/:trip_id/expenses
Edit trip (from dashboard)	PUT /api/trips/: trip_id
Edit/delete expense (from all expenses page)	PUT /api/expenses/:expense_id DELETE /api/expenses/:expense_id
*Share trip	POST /api/trips/:trip_id/share

3.1 Flow

- The login/register will store the user or client's token. All later requests will send "Authorization: Bearer <token>" so the server knows which user is logged in.
- The My Trips page will load with GET /api/trips. The user will tap on a trip and GET /api/trips/:trip_id loads the dashboard.
- The "Add expense" button on the dashboard will go to the add expense page. The user taps the "Add" button and POST /api/trips/:trip_id/expenses will create the new expense and the user is taken back to the dashboard with GET /api/trips/:trip_id to refresh the data.
- The "All expenses" button on the dashboard will take the user to the all expenses page with GET /api/trips/:trip_id/expenses.
- The "Create trip" button on the add new trip page will POST /api/trips and then open that trips dashboard with GET /api/trips/:trip_id
- *For the stretch feature users can share a trip by pressing the add user button on the edit trip page. It will be added to the trip tabe shared_user_id with POST /api/trips/:trip_id/share

3.2 RESTful Principles

- HTTP verbs are used consistently: GET/read, POST/create, PUT/update, DELETE/remove.
- Paths logically correspond to their purpose and are nested clearly and correctly (/trips/:trip_id/expenses) to show hierarchy.
- Business logic is in the service layer on GET api/trips/:trip_id so UI only displays data.
- Errors return status code and error message JSON { "error": "message" }.
- Nouns are used in URL not verbs (/api/trips)